

AI On Old Linux Laptop

Yogesh Haribhau Kulkarni

AIOOLL Workshop

Workshop Agenda

Hour 1: Setup & Foundations

00:00 What is AIOOLL?

00:10 Install Ubuntu on old laptop

00:25 System prep & Miniconda

00:35 Ollama install & model pull

00:45 Clone repo & bootstrap

00:55 Module 1: Classical ML

Hour 2: AI Modules

01:10 Module 2: Deep Learning (PyTorch)

01:25 Module 3: RAG Pipeline

01:40 Module 4: AI Agents

01:50 Module 5: Computer Vision

01:55 Module 6: LLM Inference

02:00 Tests & Next Steps

Hardware You Need

Intel i3 laptop (any generation) · 8 GB RAM · SSD (strongly preferred) · USB drive (8 GB+)

- **What You Will Build Today:** Six working AI applications: each with a Python driver, pytest test suite, and a Streamlit UI: all offline, all on your laptop.

Real Benchmark: Intel i3-8130U, 8 GB RAM, SSD
Gemma2:2b chat: ~8 tok/s · RAG query: ~50 ms · ML train: <1 s · CV face detect: 15 FPS

AIOOLL: Six Modules at a Glance

Module	Tech	Task	Time
1. Classical ML	scikit-learn	Spam detect, price predict	<1 s
2. Deep Learning	PyTorch (CPU)	LSTM sentiment, MLP regression	~30 s/ epoch
3. RAG Pipeline	LangChain + ChromaDB	Document Q&A, local KB	~50 ms/ query
4. AI Agents	LangGraph + Ollama	Research & code review agents	~5–15 s
5. Computer Vision	OpenCV	Face detect, motion detect	15 FPS
6. LLM Inference	Ollama	Benchmark, chat, prompting	~8 tok/s

Every module provides:

- **driver.py:** headless runner
- **tests/test_*.py:** pytest suite
- **ui/app.py:** Streamlit dashboard
- Bundled dataset (no download)

Part 1: Install Ubuntu

Prerequisite: Free your hardware from Windows

Why Xubuntu?

Choosing the Right Flavour

- **Standard Ubuntu (GNOME):** ~800 MB idle RAM: leaves only 7.2 GB for AI workloads on an 8 GB machine. **Avoid.**

- **Xubuntu (XFCE):** ~280 MB idle RAM: recommended. Lightweight, stable, full hardware support.
- **Lubuntu (LXQt):** ~200 MB idle: slightly lighter, less polished. Good alternative.
- **Version:** Ubuntu 22.04 LTS (Jammy Jellyfish): supported until April 2027. Stable Python 3.10 available natively.
- **Download Xubuntu ISO:** xubuntu.org/download (choose 22.04 LTS, 64-bit)

RAM Savings Summary

GNOME 800 MB → XFCE 280 MB = **520 MB freed**: roughly the size of a quantized 500M-parameter model

Step 1: Create a Bootable USB Drive

- **Tool needed:** Rufus (Windows): free, at rufus.ie
- **USB drive:** 8 GB minimum, all data will be erased

Procedure in Rufus:

1. Plug in USB drive
2. Open Rufus → *Device*: select your USB drive
3. *Boot selection*: click **SELECT** → choose downloaded **xubuntu-22.04-desktop-amd64.iso**
4. *Partition scheme*: **GPT** (modern laptops) or **MBR** (very old BIOS)
5. *File system*: FAT32
6. Click **START** → accept ISO mode warning → wait ~5 minutes
7. Click **CLOSE** when done

Important

Back up all Windows data before proceeding. The next steps will modify your disk.

Step 2: Boot from USB & Install Xubuntu

Enter BIOS/UEFI boot menu:

- Restart laptop → press **F12** (Dell/Lenovo/HP) or **F2/Esc/Del** (varies by brand)
- Select USB drive from boot menu

Xubuntu Installer: key choices:

1. **Try or Install:** Choose *Install Xubuntu*
2. **Keyboard layout:** Select your language
3. **Updates & other software:** Tick “Install third-party software” (codecs, drivers)

4. Installation type:

- *Erase disk and install Xubuntu*: full replacement (simplest, recommended for this workshop)
- *Install alongside Windows*: dual boot (keep Windows)

5. **Timezone, name, password**: Fill in and note your password

6. Wait ~10-20 minutes → **Restart Now** when prompted → remove USB

After Reboot

Login with your password. You now have Xubuntu 22.04. Open a terminal: **Ctrl+Alt+T**

Step 3: First Boot: Update & Essential Tools

```
# Update package lists and upgrade all packages
sudo apt update && sudo apt upgrade -y

# Install essential build tools
sudo apt install -y build-essential curl wget git \
    httpd btop lm-sensors python3-pip

# Check sensors (CPU temperature monitoring)
sudo sensors-detect --auto
sensors

# Set CPU to performance mode (run before AI workloads)
echo performance | sudo tee \
    /sys/devices/system/cpu/cpu*/cpufreq/scaling_governor

# Verify Python version (should be 3.10)
python3 --version
```

Step 4: Install Miniconda

```
# Download Miniconda installer (Linux 64-bit)
wget https://repo.anaconda.com/miniconda/
Miniconda3-latest-Linux-x86_64.sh \
    -O ~/miniconda.sh

# Run installer (-b = batch, -p = prefix path)
bash ~/miniconda.sh -b -p $HOME/miniconda3

# Initialise conda for bash
~/miniconda3/bin/conda init bash

# Reload shell (or open new terminal)
source ~/.bashrc
```

```
# Verify installation
conda --version      # should show: conda
24.x.x
conda info | head -5
```

Why Conda?

Conda manages Python versions per environment, isolates packages, and handles binary dependencies (numpy, torch) more reliably than pip alone on Linux.

Part 2: AIOOLL Setup

Clone the repo, create the environment, pull models

Clone Repo & Create Conda Environment

```
# Clone the AIOOLL repository
git clone
https://github.com/yourusername/AIOOLL.git
cd AIOOLL

# Create conda environment with Python 3.10,
# activate it, and install all dependencies
conda create -n aiooll python=3.10 -y \
    && conda activate aiooll \
    && pip install -r requirements.txt

# Run the full bootstrap script
# (downloads NLTK data + OpenCV Haar cascades)
bash scripts/setup.sh

# Verify the environment
conda activate aiooll
python -c "import sklearn, torch, cv2, streamlit; \
    print('All core packages OK!)"
python -c "import torch; \
    print(f'PyTorch {torch.__version__}, CPU
    only!)"
```

Install Ollama & Pull Quantized Models

```
# Install Ollama (official installer)
curl -fsSL https://ollama.ai/install.sh | sh

# Start Ollama server in background
ollama serve &

# Pull quantized CPU-friendly models
# Gemma2 2B ~1.5 GB -- best for chat and RAG
ollama pull gemma2:2b
```

```
# Qwen2 1.5B ~0.9 GB -- fastest, multilingual
ollama pull qwen2:1.5b

# Embedding model for RAG ~270 MB
ollama pull nomic-embed-text

# Verify models are ready
ollama list
```

Disk Space

All three models together use ≈2.7 GB. Ensure at least 5 GB free on your SSD before pulling.

Repository Structure: Quick Map

```
AIOOLL/
src/
  ml/          # Module 1: scikit-learn
  |-- driver.py # run with: python driver.py
  |-- ui/app.py # run with: streamlit run
  ui/app.py
  |-- tests/   # run with: pytest tests/ -v
  |-- data/    # sms_spam.csv, house_prices.csv
  dl/          # Module 2: PyTorch
  rag/         # Module 3: LangChain + ChromaDB
  agents/      # Module 4: LangGraph
  cv/          # Module 5: OpenCV
  llm_inference/ # Module 6: Ollama benchmarking
  tests/       # Integration tests (run from
    root)
scripts/
  setup.sh     # bootstrap (run once)
  pull_models.sh # pull all Ollama models
  Makefile     # make run-ml / make ui-rag / etc.
```

Makefile: Your Shortcut Reference

```
# Always activate the environment first
conda activate aiooll

make setup      # install deps + NLTK + CV
  cascades
make models     # pull all Ollama models

# Run headless drivers
make run-ml     # cd src/ml && python driver.py
make run-dl     # cd src/dl && python driver.py
make run-rag    # cd src/rag && python driver.py
make run-agents # cd src/agents && python
  driver.py
make run-cv     # cd src/cv && python driver.py
make run-llm    # cd src/llm_inference && python
  driver.py
```

```
# Launch Streamlit UIs (opens browser
  automatically)
make ui-ml      # streamlit run src/ml/ui/app.py
make ui-rag     # streamlit run src/rag/ui/app.py
make ui-agents  # streamlit run
                src/agents/ui/app.py

make test      # pytest all modules
make clean     # remove all generated artifacts
```

Module 1: Classical Machine Learning

src/ml/driver.py · scikit-learn · CPU <1s

Module 1: ML Concepts

- **Task A: Spam Detection:** Binary text classification using TF-IDF features fed into four algorithms: Naive Bayes, Logistic Regression, Linear SVM, Random Forest. Algorithms compete; best AUC-ROC wins.
- **Task B: House Price Prediction:** Tabular regression on 8 engineered features (area, age, distance, amenities). Ridge Regression vs. Gradient Boosting compared on R^2 , RMSE, MAE.
- **TF-IDF:** Term Frequency $\times$ Inverse Document Frequency. Converts raw text to a numerical matrix where common words are down-weighted. `ngram_range=(1,2)` captures both single words and two-word phrases.
- **Pipeline:** scikit-learn Pipeline chains vectorizer + classifier so the same transformations are applied consistently to train and test splits: preventing data leakage.
- **Cross-Validation:** 5-fold stratified CV gives an unbiased estimate of generalisation performance by training/testing on 5 different splits.

Run It

cd src/ml && python driver.py then make ui-ml

Module 1: ML Code: Imports & Paths

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from pathlib import Path
from sklearn.model_selection import (
    train_test_split, cross_val_score,
    StratifiedKFold
```

```
)
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler,
    LabelEncoder
from sklearn.feature_extraction.text import
    TfidfVectorizer
from sklearn.metrics import (
    classification_report, roc_auc_score,
    mean_squared_error, r2_score,
    mean_absolute_error
)
from sklearn.naive_bayes import MultinomialNB
from sklearn.linear_model import
    LogisticRegression, Ridge
from sklearn.svm import LinearSVC
from sklearn.ensemble import (
    RandomForestClassifier,
    GradientBoostingRegressor
)
import joblib
from loguru import logger

ROOT = Path(__file__).parent
DATA_DIR = ROOT / "data"
MODELS_DIR = ROOT / "models";
    MODELS_DIR.mkdir(exist_ok=True)
RESULTS_DIR = ROOT / "results";
    RESULTS_DIR.mkdir(exist_ok=True)
```

Module 1: ML Code: SpamClassifier (Part 1)

```
class SpamClassifier:
    ALGORITHMS = {
        "Naive Bayes": Pipeline([
            ("tfidf", TfidfVectorizer(
                ngram_range=(1, 2),
                max_features=5000,
                sublinear_tf=True)),
            ("clf", MultinomialNB(alpha=0.1))
        ]),
        "Logistic Regression": Pipeline([
            ("tfidf", TfidfVectorizer(
                ngram_range=(1, 2),
                max_features=5000,
                sublinear_tf=True)),
            ("clf", LogisticRegression(
                C=1.0, max_iter=1000,
                random_state=42))
        ]),
        "Linear SVM": Pipeline([
            ("tfidf", TfidfVectorizer(
                ngram_range=(1, 2),
                max_features=5000,
```

```
                sublinear_tf=True)),
            ("clf", LinearSVC(
                C=1.0, max_iter=1000,
                random_state=42))
        ]),
    }
    def __init__(self,
        data_path=DATA_DIR/"sms_spam.csv"):
        self.data_path = data_path
        self.results: dict = {}
        self.best_model = None
```

Module 1: ML Code: SpamClassifier (Part 2)

```
def load_data(self):
    df = pd.read_csv(
        self.data_path, encoding="utf-8",
        quotechar='"')
    df["label_num"] = (df["label"] ==
        "spam").astype(int)
    return df

def run(self):
    df = self.load_data()
    X, y = df["text"], df["label_num"]
    X_train, X_test, y_train, y_test =
        train_test_split(
            X, y, test_size=0.2, random_state=42,
            stratify=y)
    kf = StratifiedKFold(n_splits=5,
        shuffle=True,
                                random_state=42)

    best_auc = 0
    for name, pipeline in
        self.ALGORITHMS.items():
        pipeline.fit(X_train, y_train)
        try:
            y_score =
                pipeline.decision_function(X_test)
            except AttributeError:
                y_score =
                    pipeline.predict_proba(X_test)[:,:1]
            auc = roc_auc_score(y_test, y_score)
            cv = cross_val_score(
                pipeline, X, y, cv=kf,
                scoring="f1")
            logger.info(f"{name}: AUC={auc:.4f} "
                f"CV-F1={cv.mean():.4f}+-{cv.std():.4f}")
            if auc > best_auc:
                best_auc = auc; self.best_model =
                    pipeline
```

```
joblib.dump(self.best_model,
MODELS_DIR/"spam_clf.pkl")
```

Module 1: ML Code: HousePricePredictor

```
class HousePricePredictor:
    def __init__(self,
        data_path=DATA_DIR/"house_prices.csv"):
        self.data_path = data_path;
        self.best_model = None

    def feature_engineering(self, df):
        df = df.copy()
        df["total_rooms"] = df["bedrooms"] +
        df["bathrooms"]
        df["amenity_score"] = df["has_garage"] +
        df["has_garden"]
        df["age_category"] = pd.cut(
            df["age_years"], bins=[0,5,15,30,100],

        labels=["new", "recent", "old", "vintage"])
        df["age_category"] =
        LabelEncoder().fit_transform(
            df["age_category"])
        return df

    def run(self):
        df = self.feature_engineering(
            pd.read_csv(self.data_path))
        feats =
        ["area_sqft", "bedrooms", "bathrooms",
            "age_years", "distance_center_km",

        "has_garage", "has_garden", "floor_level",

        "total_rooms", "amenity_score", "age_category"]
        X, y = df[feats], df["price_lakh"]
        Xtr, Xte, ytr, yte = train_test_split(
            X, y, test_size=0.2, random_state=42)
        for name, pipe in {
            "Ridge":
            Pipeline([("sc", StandardScaler()),

        ("r", Ridge(alpha=1.0))]),

            "GBM":
            Pipeline([("sc", StandardScaler()),
                ("r", GradientBoostingRegressor(
                    n_estimators=200, learning_rate=0.05))]
            )}.items():
            pipe.fit(Xtr, ytr)
            logger.info(f"{name}
            R2={r2_score(yte, pipe.predict(Xte)):.4f}")
```

Module 1: ML Code: main() & Expected Output

```
def main():
    # --- Task 1: Spam Classification ---
    clf = SpamClassifier()
    clf.run()
    test_msgs = [
        "WIN a FREE prize! Call 0800-123456 NOW!",
        "Hey, are you coming to lunch at 1pm?",
    ]
    for msg in test_msgs:
        pred = clf.best_model.predict([msg])[0]
        label = "SPAM" if pred == 1 else "HAM"
        logger.info(f"[{label}] {msg[:50]}")

    # --- Task 2: House Price Regression ---
    pred = HousePricePredictor()
    pred.run()
    sample =
        {"area_sqft":1200, "bedrooms":3, "bathrooms":2,

        "age_years":10, "distance_center_km":5.0,

        "has_garage":1, "has_garden":1, "floor_level":2}
    logger.info(f"Predicted price:
        {pred.best_model.predict(
            pd.DataFrame([sample]))[0]:.2f} Lakh")

    if __name__ == "__main__":
        main()
```

Expected terminal output (abridged)

```
Naive Bayes: AUC=0.9750 CV-F1=0.942±0.023 [SPAM] WIN
a FREE prize! Call 0800-123456 NOW! Predicted price:
72.45 Lakh
```

Module 2: Deep Learning (PyTorch CPU)

src/dl/driver.py · Bidirectional LSTM · Tabular MLP

Module 2: DL Concepts

- **Why PyTorch on CPU?** PyTorch ships a CPU-only build (whl/cpu) that is ~2× smaller and installs cleanly without CUDA. All tensor operations and autograd work identically: just slower.
- **Bidirectional LSTM:** Each token attends to tokens both before and after it. The final hidden states of the forward and backward passes are concatenated, giving a context-aware sentence embedding. Better than unidirectional for classification.

- **Batch Normalisation:** Normalises layer inputs per mini-batch, reducing internal covariate shift. Allows higher learning rates and speeds convergence on tabular data.
- **Huber Loss:** Combines MSE (for small errors) and MAE (for large errors), making regression robust to outliers in the house price dataset.
- **CPU thread tuning:** `torch.set_num_threads(4)` pins PyTorch to physical cores; hyperthreads often slow matrix multiply on old CPUs.

Run It

```
cd src/dl && python driver.py then make ui-dl
```

Module 2: DL Code: Dataset & LSTM Model

```
import torch, torch.nn as nn, torch.optim as optim
from torch.utils.data import Dataset, DataLoader,
TensorDataset

torch.set_num_threads(4) # physical cores only

class SentimentDataset(Dataset):
    TEXTS = [
        ("I love this product! Works perfectly.",
        1),
        ("Terrible quality, broke after one day.",
        0),
        # ... 22 more reviews (see driver.py) ...
    ]
    def __init__(self):
        self.vocab = {"<pad>":0, "<unk>":1}
        for text, _ in self.TEXTS:
            for t in text.lower().split():
                w = " ".join(c for c in w if
                    c.isalpha())
                if w and w not in self.vocab:
                    self.vocab[w] = len(self.vocab)
        self.max_len = 15
    def encode(self, text):
        ids = [self.vocab.get(
            " ".join(c for c in t if c.isalpha()), 1)
            for t in text.lower().split()]
        return ids[:self.max_len] + \
            [0]*max(0, self.max_len - len(ids))
    def __len__(self): return len(self.TEXTS)
    def __getitem__(self, i):
        t, l = self.TEXTS[i]
        return
        (torch.tensor(self.encode(t), dtype=torch.long),
        torch.tensor(l, dtype=torch.float))
```

Module 2: DL Code: LSTM Architecture

```
class LSTMClassifier(nn.Module):
    """Bidirectional LSTM for binary text
    classification."""
    def __init__(self, vocab_size, embed_dim=32,
                  hidden_dim=64, dropout=0.3):
        super().__init__()
        self.embedding = nn.Embedding(
            vocab_size, embed_dim, padding_idx=0)
        self.lstm = nn.LSTM(
            embed_dim, hidden_dim,
            batch_first=True, bidirectional=True,
            dropout=dropout)
        self.dropout = nn.Dropout(dropout)
        self.classifier = nn.Sequential(
            nn.Linear(hidden_dim * 2, 32),
            nn.ReLU(),
            nn.Dropout(dropout),
            nn.Linear(32, 1),
            nn.Sigmoid()
        )
    def forward(self, x):
        emb = self.dropout(self.embedding(x))
        _, (hidden, _) = self.lstm(emb)
        # concat forward + backward final hidden
        states
        ctx = torch.cat([hidden[-2], hidden[-1]],
            dim=1)
        return self.classifier(ctx).squeeze(1)
```

Module 2: DL Code: Training Loop

```
def train_lstm_classifier(epochs=30):
    dataset = SentimentDataset()
    loader = DataLoader(dataset, batch_size=8,
        shuffle=True)
    model =
        LSTMClassifier(vocab_size=len(dataset.vocab))
    opt = optim.Adam(model.parameters(),
        lr=1e-3,
        weight_decay=1e-4)
    loss_fn = nn.BCELoss()
    sched = optim.lr_scheduler.StepLR(
        opt, step_size=10, gamma=0.5)

    for epoch in range(epochs):
        model.train()
        epoch_loss, correct, total = 0.0, 0, 0
        for Xb, yb in loader:
            opt.zero_grad()
            preds = model(Xb)
            loss = loss_fn(preds, yb)
            loss.backward()
```

```
nn.utils.clip_grad_norm_(model.parameters(),
    1.0)
    opt.step()
    epoch_loss += loss.item()
    correct +=
        ((preds>0.5)==yb.bool()).sum().item()
    total += len(yb)
    sched.step()
    if (epoch+1) % 10 == 0:
        logger.info(f"Epoch {epoch+1} "
            f"Loss={epoch_loss/len(loader):.4f} "
            f"Acc={correct/total:.3f}")
    torch.save(model.state_dict(),
        "models/lstm.pt")
    return model
```

Module 2: DL Code: Tabular MLP

```
class TabularMLP(nn.Module):
    """MLP with BatchNorm for house price
    regression."""
    def __init__(self, input_dim):
        super().__init__()
        self.network = nn.Sequential(
            nn.Linear(input_dim, 128),
            nn.BatchNorm1d(128), nn.ReLU(),
            nn.Dropout(0.2),
            nn.Linear(128, 64),
            nn.BatchNorm1d(64), nn.ReLU(),
            nn.Dropout(0.2),
            nn.Linear(64, 32), nn.ReLU(),
            nn.Linear(32, 1)
        )
    def forward(self, x):
        return self.network(x).squeeze(1)

def train_tabular_mlp(epochs=100):
    # load house_prices.csv, StandardScaler, split
    # ... (see full driver.py)
    model = TabularMLP(input_dim=8)
    opt = optim.AdamW(model.parameters(),
        lr=1e-3, weight_decay=1e-3)

    for epoch in range(epochs):
        model.train()
        for Xb, yb in loader:
            opt.zero_grad()
            loss = nn.HuberLoss()(model(Xb), yb)
            loss.backward(); opt.step()

    # compute R-squared on test set
    # logger.info(f"Test R2: {r2:.4f}")
    torch.save(model.state_dict(), "models/mlp.pt")
```

Module 3: RAG Pipeline

src/rag/driver.py · LangChain · ChromaDB · Ollama

Module 3: RAG Concepts

- **RAG = Retrieve-then-Generate:** Instead of asking an LLM to answer from memory, we first retrieve the most relevant passages from a document store and include them in the prompt as context.
- **Embeddings:** `nomic-embed-text` converts text chunks to 768-dimensional vectors (via Ollama, CPU). Semantically similar texts have nearby vectors.
- **ChromaDB:** A lightweight vector database that stores embeddings on disk (`.chroma.db/`). At query time, it computes cosine similarity to find the top-K most relevant chunks in milliseconds.
- **Chunking:** `RecursiveCharacterTextSplitter` splits documents into 500-character chunks with 50-character overlap so that no important sentence is cut at a boundary.
- **RetrievalQA Chain:** LangChain's `RetrievalQA` wires retriever → prompt template → Ollama LLM into one callable.
- **Offline:** Index is built once and persisted. Subsequent runs skip re-embedding.

Run It

Start Ollama first: `ollama serve` & then make `run-rag`

Module 3: RAG Code: Config & Prompt

```
from langchain.text_splitter import
    RecursiveCharacterTextSplitter
from langchain_community.document_loaders import
    TextLoader
from langchain_community.vectorstores import Chroma
from langchain_community.embeddings import
    OllamaEmbeddings
from langchain_community.llms import Ollama
from langchain.chains import RetrievalQA
from langchain.prompts import PromptTemplate
from langchain.schema import Document

EMBED_MODEL = "nomic-embed-text"
LLM_MODEL = "gemma2:2b"
CHUNK_SIZE = 500
CHUNK_OVERLAP = 50
TOP_K = 3

RAG_PROMPT = PromptTemplate(
    input_variables=["context", "question"],
    template="""Answer ONLY from the context below.
    If unsure, say "I don't have that information."""
```



```
Context:
{context}

Question: {question}

Answer: ""
```

Module 3: RAG Code: Build Index

```
class LocalRAGPipeline:
    def __init__(self, embed_model=EMBED_MODEL,
                 llm_model=LLM_MODEL,
                 kb_dir=Path("knowledge_base"),
                 persist_dir=Path(".chroma_db")):
        self.embed_model = embed_model
        self.llm_model = llm_model
        self.kb_dir = kb_dir
        self.persist_dir = persist_dir
        self.vectorstore = None
        self.qa_chain = None

    def build_index(self, force_rebuild=False):
        db_file = self.persist_dir /
            "chroma.sqlite3"
        embeddings =
            OllamaEmbeddings(model=self.embed_model)
        if not force_rebuild and db_file.exists():
            self.vectorstore = Chroma(

            persist_directory=str(self.persist_dir),
            embedding_function=embeddings)
        return
        docs = [TextLoader(str(f),
            encoding="utf-8").load()[0]
            for f in self.kb_dir.glob("*.txt")]
        splitter = RecursiveCharacterTextSplitter(
            chunk_size=CHUNK_SIZE,
            chunk_overlap=CHUNK_OVERLAP)
        chunks = splitter.split_documents(docs)
        self.vectorstore = Chroma.from_documents(
            chunks, embeddings,

        persist_directory=str(self.persist_dir))
        self.vectorstore.persist()
        logger.info(f"Indexed {len(chunks)}
            chunks")

    def build_chain(self):
```

Module 3: RAG Code: Build Chain & Query

```
def build_chain(self):
```

```
llm = Ollama(model=self.llm_model,
             temperature=0.1,
             num_predict=512)
retriever = self.vectorstore.as_retriever(
    search_type="similarity",
    search_kwargs={"k": TOP_K})
self.qa_chain =
    RetrievalQA.from_chain_type(
        llm=llm,
        chain_type="stuff",
        retriever=retriever,
        chain_type_kwargs={"prompt":
            RAG_PROMPT},
        return_source_documents=True)

def query(self, question: str) -> dict:
    import time
    t0 = time.time()
    result = self.qa_chain({"query": question})
    return {
        "answer": result["result"],
        "sources":
            [d.metadata.get("source", "?")
             for d in
             result["source_documents"]],
        "latency": round(time.time()-t0, 2)
    }

# Usage
pipeline = LocalRAGPipeline()
pipeline.build_index()
pipeline.build_chain()
r = pipeline.query("Which model is best for
    multilingual use?")
print(r["answer"], r["latency"])
```

Module 4: AI Agents with LangGraph

src/agents/driver.py · State machines · Local LLM tools

Module 4: Agent Concepts

- **LangGraph:** Builds agents as typed state machines (directed graphs). Each node is a Python function that reads/writes a shared TypedDict state. Edges define the flow: fixed or conditional.
- **ResearchAgent: Linear Graph:** START → plan → research → synthesize → END. Each node calls the local Ollama LLM with a focused prompt.
- **CodeReviewAgent: Conditional Graph:** After syntax_check, a router reads the result and branches to pass_review, minor_review, or major_review.

- **Tools:** LangChain @tool decorators expose Python functions (calculator, word counter, AST syntax checker) that agents can call without an LLM round-trip.
- **No OpenAI API key needed:** All LLM calls go to localhost:11434 (Ollama).

Run It

make run-agents then make ui-agents

Module 4: Agent Code: Tools

```
import math, json
from langchain.tools import tool
from langgraph.graph import StateGraph, END
from langchain_community.llms import Ollama
from typing import TypedDict

@tool
def calculator(expression: str) -> str:
    """Safely evaluate a math expression."""
    allowed = {
        "abs":abs,"round":round,"sqrt":math.sqrt,

        "pow":pow,"log":math.log,"pi":math.pi,"e":math.e,

        "sin":math.sin,"cos":math.cos,"tan":math.tan,
    }
    try:
        result =
            eval(expression,{"_builtins_":{}}),allowed)
        return f"Result: {result}"
    except Exception as ex:
        return f"Error: {ex}"

@tool
def python_syntax_checker(code: str) -> str:
    """Check Python code for syntax errors via
        ast.parse."""
    import ast
    try:
        tree = ast.parse(code)
        return json.dumps({
            "status":"valid",
            "functions": sum(1 for n in
                ast.walk(tree)
                    if
                    isinstance(n,ast.FunctionDef)),
            "lines": len(code.splitlines())})
    except SyntaxError as e:
        return
        json.dumps({"status":"error","msg":str(e),
            "line":e.lineno})
```

Module 4: Agent Code: ResearchAgent Graph

```
class ResearchState(TypedDict):
    topic: str; research_plan: str
    gathered_info: str; final_report: str;
    step_count: int

class ResearchAgent:
    def __init__(self, model="gemma2:2b"):
        self.llm = Ollama(model=model,

        temperature=0.3,num_predict=400)
        self.graph = self._build_graph()

    def _plan(self, s: ResearchState) ->
    ResearchState:
        plan = self.llm.invoke(
            f"Write a 3-point research plan for:
            {s['topic']}")
        return
        {**s,"research_plan":plan,"step_count":s["step_count"]+1}

    def _research(self, s: ResearchState) ->
    ResearchState:
        info = self.llm.invoke(
            f"Topic: {s['topic']}\nPlan:
            {s['research_plan']}\n"
            "Provide 3 key facts. Under 150
            words.")
        return
        {**s,"gathered_info":info,"step_count":s["step_count"]+1}

    def _synthesize(self, s: ResearchState) ->
    ResearchState:
        rep = self.llm.invoke(
            f"Synthesize into a 2-paragraph
            report:\n{s['gathered_info']}")
        return
        {**s,"final_report":rep,"step_count":s["step_count"]+1}

    def _build_graph(self):
        wf = StateGraph(ResearchState)
        wf.add_node("plan", self._plan)
        wf.add_node("research", self._research)
        wf.add_node("synthesize", self._synthesize)
        wf.set_entry_point("plan")
        wf.add_edge("plan","research")
        wf.add_edge("research","synthesize")
        wf.add_edge("synthesize", END)
        return wf.compile()
```

Module 4: Agent Code: CodeReview Conditional Graph

```
class CodeReviewState(TypedDict):
    code: str; syntax_check: str
    severity: str; review_feedback: str;
    step_count: int

class CodeReviewAgent:
    def __init__(self, model="gemma2:2b"):
        self.llm =
        Ollama(model=model,temperature=0.2)
        self.graph = self._build_graph()

    def _syntax(self, s):
        return
        {**s,"syntax_check":python_syntax_checker.invoke(s["code"]),
        "step_count":s["step_count"]+1}

    def _route(self, s) -> str:
        data = json.loads(s["syntax_check"])
        if data["status"]=="syntax_error": return
        "major"
        if data.get("functions",0)==0: return
        "minor"
        return "pass"

    def _pass_review(self, s):
        fb = self.llm.invoke(f"Good code,
        2-sentence review:\n{s['code']}[:400]")
        return
        {**s,"severity":"pass","review_feedback":fb}

    def _major_review(self, s):
        fb = self.llm.invoke(f"Syntax error
        {s['syntax_check']}. \nFix:\n{s['code']}[:400]")
        return
        {**s,"severity":"major","review_feedback":fb}

    def _build_graph(self):
        wf = StateGraph(CodeReviewState)
        for name,fn in [("chk",self._syntax),
        ("pass",self._pass_review),

        ("major",self._major_review)]:
            wf.add_node(name, fn)
            wf.set_entry_point("chk")
            wf.add_conditional_edges("chk",
            self._route,

            {
                "pass": "pass",
                "minor": "pass",
                "major": "major"
            })
            for n in ["pass", "major"]: wf.add_edge(n,
            END)
        return wf.compile()
```

Module 5: Computer Vision (OpenCV)

src/cv/driver.py · Haar Cascades · MOG2 Background Subtraction

Module 5: CV Concepts

- **Haar Cascade:** A sliding-window classifier trained on thousands of face/non-face images. Uses Haar-like features (differences of rectangular sums). Cascade architecture rejects non-face regions early: very fast on CPU (~15 FPS).
- **MOG2 (Mixture of Gaussians v2):** Models each pixel's intensity as a mixture of Gaussians learned from recent frames. Pixels deviating from the background distribution are flagged as foreground (motion). Shadow detection built-in.
- **Canny Edge Detection:** Two-threshold hysteresis edge detector. Pixels above threshold2 are strong edges; pixels between the thresholds are kept only if connected to a strong edge.
- **Laplacian Variance (sharpness):** The variance of the Laplacian of an image measures the spread of edge strengths. Low variance $\Rightarrow$ blurry image.
- **No GPU, no DL:** All algorithms are classical signal processing: run at full speed on a 2012 i3 CPU.

Run It

make run-cv then streamlit run src/cv/ui/app.py

Module 5: CV Code: Face Detector

```
import cv2
import numpy as np
from pathlib import Path

MODELS_DIR = Path(__file__).parent / "models"

class HaarFaceDetector:
    def __init__(self):
        cascade =
        MODELS_DIR/"haarcascade_frontalface_default.xml"
        if not cascade.exists():
            # fall back to OpenCV bundled cascades
            cascade =
            Path(cv2.__file__).parent/"data"/cascade.name
            self.face_cascade =
            cv2.CascadeClassifier(str(cascade))

    def detect(self, image: np.ndarray):
        gray = cv2.cvtColor(image,
        cv2.COLOR_BGR2GRAY) \
        if image.ndim == 3 else image
```

```

gray = cv2.equalizeHist(gray)
faces = self.face_cascade.detectMultiScale(
    gray, scaleFactor=1.1, minNeighbors=5,
    minSize=(30, 30),
    flags=cv2.CASCADE_SCALE_IMAGE)
return list(faces) if len(faces) > 0 else
[]

def annotate(self, image, faces):
    out = image.copy()
    for (x, y, w, h) in faces:
        cv2.rectangle(out, (x,y), (x+w,y+h), (0,255,0), 2)
        cv2.putText(out, "Face", (x,y-8),
            cv2.FONT_HERSHEY_SIMPLEX, 0.6, (0,255,0), 2)
    return out

```

Module 5: CV Code: Motion Detector & Image Analyser

```

class MotionDetector:
    def __init__(self, history=500,
        threshold=16.0):
        self.bg =
            cv2.createBackgroundSubtractorMOG2(
                history=history,
                varThreshold=threshold,
                detectShadows=True)
        self.kernel = cv2.getStructuringElement(
            cv2.MORPH_ELLIPSE, (5, 5))

    def process_frame(self, frame):
        mask = self.bg.apply(frame)
        _, mask = cv2.threshold(mask, 200, 255,
            cv2.THRESH_BINARY)
        mask =
            cv2.morphologyEx(mask, cv2.MORPH_OPEN, self.kernel)
        mask = cv2.dilate(mask, self.kernel,
            iterations=2)
        contours, _ = cv2.findContours(
            mask, cv2.RETR_EXTERNAL,
            cv2.CHAIN_APPROX_SIMPLE)
        moving = [c for c in contours
            if cv2.contourArea(c) > 500]
        ratio = sum(cv2.contourArea(c) for c in
            moving) \
            / (frame.shape[0]*frame.shape[1])
        return mask, moving, ratio

class ImageAnalyzer:
    def analyze(self, image):
        gray =
            cv2.cvtColor(image, cv2.COLOR_BGR2GRAY) \

```

```

        if image.ndim==3 else image
        edges = cv2.Canny(gray, 50, 150)
        lap_var = cv2.Laplacian(gray,
            cv2.CV_64F).var()
        hist =
            cv2.calcHist([gray], [0], None, [256], [0,256])
        return {"sharpness":
            round(float(lap_var), 2),
            "edge_density":
            round(float(edges.sum()/(255*gray.size)), 4),
            "is_blurry": lap_var < 100}

```

Module 6: Local LLM Inference & Benchmarking

src/llm_inference/driver.py · Ollama REST API · Prompt Engineering

Module 6: LLM Inference Concepts

- **Ollama REST API:** Ollama exposes `http://localhost:11434` with endpoints: `/api/generate` (completion), `/api/chat` (multi-turn), `/api/embeddings`, `/api/tags` (list models).
- **Tokens per second (tok/s):** The primary CPU inference speed metric. On i3-8130U: Gemma2:2b $\approx$ 8 tok/s, Qwen2:1.5b $\approx$ 12 tok/s, TinyLlama $\approx$ 15 tok/s.
- **Prompt Engineering Patterns:**
 - *Zero-shot:* Direct question, no examples
 - *Few-shot:* Provide 2–3 input/output examples before the query
 - *Chain-of-thought:* Ask the model to reason step-by-step before answering
 - *Structured output:* Instruct JSON-only response, parse with `json.loads`
- **Temperature:** 0.0–0.2 for deterministic tasks (code, JSON); 0.7–0.9 for creative tasks.

Run It

make run-llm then make ui-llm

Module 6: LLM Code: Ollama Client

```

import httpx, time, json

OLLAMA_BASE = "http://localhost:11434"

class OllamaClient:
    def __init__(self):

```

```

        self.client = httpx.Client(timeout=120.0)

    def is_running(self) -> bool:
        try:
            return self.client.get(
                f"{OLLAMA_BASE}/api/tags").status_code==200
        except Exception: return False

    def list_models(self) -> list:
        r =
            self.client.get(f"{OLLAMA_BASE}/api/tags")
        return r.json().get("models", [])

    def generate(self, model, prompt,
        temperature=0.7, max_tokens=512)
    -> dict:
        payload = {"model":model, "prompt":prompt,
            "stream":False,

            "options":{"temperature":temperature,

            "num_predict":max_tokens}}
        t0 = time.time()
        r = self.client.post(
            f"{OLLAMA_BASE}/api/generate",
            json=payload)
        elapsed = time.time()-t0
        data = r.json()
        return {"response":
            data.get("response", ""),
            "latency_s": round(elapsed, 2),
            "tok_per_s": round(

            data.get("eval_count", 0)/max(elapsed, 0.001), 1)}

```

Module 6: LLM Code: Benchmarker & Prompt Patterns

```

PROMPTS = [
    "Explain attention mechanism in 2 sentences.",
    "Write a Python function to check if n is prime.",
    "List 5 Linux commands for system monitoring.",
]

class ModelBenchmarker:
    def __init__(self):
        self.client = OllamaClient()
        self.models = [m["name"]
            for m in
            self.client.list_models()]

    def run(self, n_prompts=3):

```



```

        for model in self.models:
            times = []
            for prompt in PROMPTS[:n_prompts]:
                r = self.client.generate(model,
                    prompt,
                    max_tokens=200)
                times.append(r["tok_per_s"])
                logger.info(f"{model}: {r['tok_per_s']} tok/s")
            logger.info(f"AVG {model}: "
                f"{sum(times)/len(times):.1f} tok/s")

class PromptEngineer:
    def __init__(self, model="gemma2:2b"):
        self.c = OllamaClient(); self.model = model

    def chain_of_thought(self, problem: str) -> str:
        prompt = (f"Solve step by step:\n{problem}\n\n"
            "Step 1:")
        return self.c.generate(self.model, prompt,
            temperature=0.3)["response"]

    def structured_json(self, task, schema) -> dict:
        prompt = f"{task}\nRespond ONLY as JSON:\n{schema}"
        r = self.c.generate(self.model, prompt,
            temperature=0.1)
        return json.loads(r["response"].strip())

```

Integration Tests & Running the Full Suite

tests/test_integration.py · pytest · No Ollama required

Tests: What Is Checked

- **TestRepoStructure:** Parametrised: asserts every directory and file listed in the README actually exists on disk. Catches copy-paste errors immediately.
- **TestDatasets:** Loads both CSVs with pandas, checks row counts, column names, no null values, both class labels present.
- **TestImports:** Verifies numpy, pandas, sklearn, torch, cv2, streamlit are importable. Optional packages (langchain, chromadb, ollama) are skipped gracefully if absent.

- **TestCrossModule:** Imports each driver.py and verifies key classes/functions are callable without running training.
- **TestOfflineReadiness:** Scans driver files for hardcoded API key patterns (sk-...), checks requirements.txt has ≥10 packages, verifies Makefile targets exist.

Module-Level Tests

Each module also has its own tests/test_*.py: TestSpamClassifier, TestHousePricePredictor, TestLSTMClassifier, TestCalculatorTool, etc.

Tests: Running & Reading Output

```

conda activate aiooll

# Run all integration tests (no Ollama needed)
pytest tests/ -v --timeout=60

# Run module-level ML tests
pytest src/ml/tests/ -v --timeout=120

# Run DL tests (slow tests need flag)
pytest src/dl/tests/ -v -m "not slow" --timeout=60

# Run agent tool tests (no LLM needed)
pytest src/agents/tests/ -v --timeout=30

# Run everything at once
make test

# Check coverage (optional)
pytest tests/ src/ml/tests/ --cov=src \
    --cov-report=term-missing

```

Expected output (abridged)

```

PASSED tests/test_integration.py::TestRepoStructure:...
PASSED tests/test_integration.py::TestDatasets::test_sms_spam
PASSED src/ml/tests/test_ml.py::TestSpamClassifier::test_auc_

```

Tests: Integration Test Code (Part 1)

```

import pytest, importlib
from pathlib import Path
ROOT = Path(__file__).parent.parent

REQUIRED_FILES = [
    "README.md", "requirements.txt", "Makefile",
    "src/ml/driver.py", "src/ml/ui/app.py",
    "src/ml/tests/test_ml.py",
    "src/ml/data/sms_spam.csv",
    "src/dl/driver.py", "src/rag/driver.py",
    "src/agents/driver.py", "src/cv/driver.py",

```

```

    "src/llm_inference/driver.py",
]

class TestRepoStructure:
    @pytest.mark.parametrize("fp", REQUIRED_FILES)
    def test_file_exists(self, fp):
        assert (ROOT/fp).is_file(), f"Missing: {fp}"

class TestDatasets:
    def test_sms_spam_loads(self):
        import pandas as pd
        df = pd.read_csv(ROOT/"src/ml/data/sms_spam.csv")
        assert len(df) >= 50
        assert set(df["label"].unique()) <= {"spam", "ham"}

    def test_house_prices_no_nulls(self):
        import pandas as pd
        df = pd.read_csv(ROOT/"src/ml/data/house_prices.csv")
        assert df.isnull().sum().sum() == 0
        assert (df["price_lakh"] > 0).all()

```

Tests: Integration Test Code (Part 2)

```

CORE_PKGS = ["numpy", "pandas", "sklearn", "torch", "cv2", "streamlit", "loguru"]

class TestImports:
    @pytest.mark.parametrize("pkg", CORE_PKGS)
    def test_importable(self, pkg):
        importlib.import_module(pkg)

    def test_torch_cpu(self):
        import torch
        x = torch.tensor([1.0, 2.0, 3.0])
        assert x.device.type == "cpu"

class TestOfflineReadiness:
    def test_no_hardcoded_api_keys(self):
        import re
        pattern = re.compile(r"sk-[A-Za-z0-9]{20,}")
        for driver in ROOT.glob("src/*/driver.py"):
            assert not pattern.search(driver.read_text()), \
                f"API key in {driver}"

    def test_makefile_targets(self):
        mf = (ROOT/"Makefile").read_text()

```

```
for t in
["setup","test","run-ml","run-rag","clean"]:
    assert f"{t}:" in mf
```

Streamlit UIs: Launching & Using

Every module has an interactive dashboard

Streamlit UI: Launch All Dashboards

```
conda activate aiooll

# ML Dashboard: spam detector + house price predictor
streamlit run src/ml/ui/app.py
# Opens: http://localhost:8501

# Deep Learning: training visualizer
streamlit run src/dl/ui/app.py
# Opens: http://localhost:8501

# RAG: document Q&A chat interface
# (requires: ollama serve & running)
streamlit run src/rag/ui/app.py

# AI Agents: thought-process visualizer
streamlit run src/agents/ui/app.py

# Computer Vision: face & motion detection
streamlit run src/cv/ui/app.py

# LLM Inference: benchmark + chat + prompt patterns
streamlit run src/llm_inference/ui/app.py
```

One UI at a time

Each `streamlit run` command occupies port 8501. Run them in separate terminals, or stop one (**Ctrl+C**) before starting another.

Hardware Optimisation for AI Workloads

Squeeze every tok/s from your old i3

Hardware Optimisation: Quick Wins

```
# 1. Set CPU governor to performance mode
echo performance | sudo tee \

    /sys/devices/system/cpu/cpu*/cpufreq/scaling_governor
cat
    /sys/devices/system/cpu/cpu0/cpufreq/scaling_governor
# should print: performance

# 2. Monitor CPU temperature (throttling check)
watch -n 2 sensors # look for Package temp < 85 C

# 3. Check available RAM before inference
free -h # aim for > 2 GB free

# 4. Drop file caches to free RAM
sudo sh -c "echo 3 > /proc/sys/vm/drop_caches"

# 5. Pin Ollama to physical cores only (2 on i3)
OMP_NUM_THREADS=2 ollama serve

# 6. Live monitoring during inference
bttop # CPU, RAM, per-core load
```

Model Selection by Available RAM

3 GB free: TinyLlama 1.1B · 4 GB: Qwen2 1.5B · 6 GB: Gemma2 2B · 8 GB: Phi-3 Mini 3.8B

Troubleshooting Common Issues

- “Ollama not running”: `ollama serve` & then wait 3 seconds and retry.
- “Model not found”: `ollama pull gemma2:2b` check disk space with `df -h`.
- **ParserError on sms_spam.csv**: Use the fixed CSV from the repo (commas in text were causing 3-field parse). Driver now uses `quotechar=''`.
- **PyTorch CUDA error**: Re-install CPU build: `pip install torch --index-url https://download.pytorch.org/whl/cpu`
- **Streamlit “module not found”**: Ensure conda env is active: `conda activate aiooll`.
- **RAM OOM during inference**: Close browser tabs, run `sudo sh -c "echo 3 > /proc/sys/vm/drop_caches"`, switch to a smaller model.
- **CV cascade not found**: Run `bash scripts/setup.sh`: it downloads Haar cascade XMLs automatically.
- **conda: command not found**: Run `source $HOME/miniconda3/bin/activate` then `conda init bash && source ~/.bashrc`.

Workshop Summary & Next Steps

What you built today:

- ✓ Xubuntu on old hardware
- ✓ Conda environment `aiooll`
- ✓ Local LLMs via Ollama
- ✓ ML: spam + price prediction
- ✓ DL: LSTM + MLP on CPU
- ✓ RAG: local document Q&A
- ✓ Agents: LangGraph state machines
- ✓ CV: face & motion detection
- ✓ All tests passing offline

Extend AIOOLL yourself:

- Add your own docs to `src/rag/knowledge.base/`
- Try `phi3:mini` (3.8B) for better reasoning
- Add a new LangGraph agent tool
- Plug webcam into the CV motion detector
- Submit benchmark results to `docs/community_benchmarks.md`
- Star the repo and share on LinkedIn!

The AIOOLL Proof

Intel i3 · 8 GB RAM · SSD · Ubuntu · Ollama = Complete AI Stack No GPU. No cloud. No credit card. Just Python and patience.

Quick Reference Card

Environment

- `conda activate aiooll`
- `ollama serve` &
- `ollama list`

Run Modules

- `make run-ml`
- `make run-dl`
- `make run-rag`
- `make run-agents`
- `make run-cv`
- `make run-llm`

Launch UIs

- `make ui-ml`
- `make ui-rag`
- `make ui-agents`

Tests

- `make test`
- `pytest tests/ -v`

- `pytest src/ml/tests/ -v`

Performance

- `echo performance | sudo tee /sys/devices/system/cpu/cpu0/cpufreq/scaling_governor`
- `free -h`

- `htop`

- `sensors`

- `Wolfsram`
- `ollama pull gemma2:2b`

- `ollama pull qwen2:1.5b`

- `ollama pull nomic-embed-text`

github.com/yogeshhk/AIOOLL